

QUIZZICAL

Project documentation and viva preparation guide

A practical guide to the application as it is implemented in this repository: its screens, state, API, persistence, project files, tests, and known limits.

Flutter application | Dart | Provider | OpenTDB | SharedPreferences

Important scope note

This report is based on the repository source and configuration. It does not treat README claims as implementation evidence. Firebase, authentication, a custom server, and a database are not present in the current project.

1. Project overview

Quizzical is a Flutter trivia application. It loads category metadata and quiz questions from the Open Trivia Database (OpenTDB), lets a player configure a quiz, and manages an active timed question session. The app stores selected preferences and progress summaries locally using SharedPreferences.

At a glance

Area	Implemented behavior
Client UI	Welcome, category browser, quiz configuration, active quiz, and result screens.
Remote data	HTTPS GET requests to OpenTDB category and question endpoints using package:http.
State	A ChangeNotifier-based QuizProvider exposed through Provider.
Local persistence	SharedPreferences for player name, quiz settings, favorites, recent categories, scores, stats, and daily challenge/streak values.
Quiz rules	1-50 requested questions, selected difficulty/type, answer shuffling, 30-second question timer, scoring and answer streaks.
Not present	Firebase, sign-in/authentication, custom backend, SQL/NoSQL database, cloud sync, and remote user profiles.

Main user flow

HomeScreen -> CategoriesScreen -> QuizConfigScreen -> QuizScreen -> ResultScreen. Category selection and configuration are held in QuizProvider. The provider requests questions through TriviaService; Question.fromJson decodes the API data and shuffles the answer options. The provider times and scores answers, and the result screen displays the final percentage.

Navigation uses Flutter MaterialPageRoute calls in the screens. The project does not configure a separate routing package or named-route table.

External service status

OpenTDB is an external API, not a project-owned backend. The application needs internet connectivity to load categories/questions. API availability and question inventory depend on OpenTDB.

2. Project structure and file inventory

Repository tree (grouped to keep generated platform boilerplate compact)

```
quizzical/  
|-- lib/  
| |-- main.dart  
| |-- models/ (category.dart, question.dart)  
| |-- providers/ (quiz_provider.dart)  
| |-- screens/ (home_screen.dart, categories_screen.dart,  
| | quiz_config_screen.dart, quiz_screen.dart, result_screen.dart)  
| |-- services/ (trivia_service.dart)  
| |-- utils/ (app_colors.dart, category_helper.dart, html_decoder.dart)  
| `-- widgets/ (answer_option.dart, category_card.dart,  
| category_illustrations.dart, illustrations.dart, primary_button.dart)  
|-- assets/ (branding/, categories/, images/, quiz/, result/)  
|-- test/ (quiz_test.dart, widget_test.dart)  
|-- android/, ios/, web/, linux/, macos/, windows/  
|-- pubspec.yaml, pubspec.lock, analysis_options.yaml, README.md, .gitignore
```

Dart source files

File	Purpose and key responsibility
lib/main.dart	Entry point; sets system UI styling, builds Material 3 theme, registers QuizProvider and opens HomeScreen.
lib/screens/home_screen.dart	Welcome screen, editable player name dialog, welcome illustration, and navigation to categories.
lib/screens/categories_screen.dart	Fetches/displays categories; client-side search, All/Favorites/Recently Played/A-Z filters, category selection and daily challenge entry.
lib/screens/quiz_config_screen.dart	Question-count slider, difficulty and question-type controls; starts quiz and presents retry/error feedback.
lib/screens/quiz_screen.dart	Question, countdown, progress, streak, answer choices, exit confirmation, and transition to results.
lib/screens/result_screen.dart	Score percentage and success/encouragement state; replay and category-selection actions.
lib/providers/quiz_provider.dart	Central ChangeNotifier state, API orchestration, question/timer/scoring logic, local preference persistence, favorites, recent history and stats.
lib/services/trivia_service.dart	OpenTDB HTTPS category/question requests, query construction, timeout handling and API/network error translation.
lib/models/category.dart	Category id/name and JSON conversion.
lib/models/question.dart	Question fields, API JSON conversion, HTML decoding and randomized answer order.
lib/utils/app_colors.dart	Shared colors for theme, feedback and category artwork surfaces.
lib/utils/category_helper.dart	OpenTDB category names, visual metadata, fallback mappings and uniqueness check.
lib/utils/html_decoder.dart	Decodes named, decimal and hexadecimal HTML entities from trivia text.
lib/widgets/primary_button.dart	Reusable filled/outlined CTA with optional icon and loading state.
lib/widgets/answer_option.dart	Reusable answer card with unselected, selected, correct and incorrect visual states.
lib/widgets/category_card.dart	Category grid tile with visual, favorite action, recent marker and score/popularity badge.
lib/widgets/category_illustrations.dart	Loads category image based on CategoryHelper mapping and falls back to a Material icon.
lib/widgets/illustrations.dart	Welcome, configuration, high-score and low-score illustrations with fallbacks.

3. Features and implementation details

Category browsing

CategoriesScreen schedules `fetchCategories()` after the first frame. The provider caches a non-empty category list unless `forceRefresh` is requested. Search and list filtering happen in the client. Favorites and recent IDs come from QuizProvider. Recently played history is updated when a quiz is completed and limited to five entries.

Quiz configuration and question retrieval

The configuration screen changes question count (1 through 50), difficulty (Any Difficulty, Easy, Medium, Hard) and type (multiple or boolean). TriviaService builds an HTTPS request to `opentdb.com/api.php` with amount and any applicable category, difficulty and type parameters. A 12-second timeout is set. The category list endpoint is `/api_category.php`.

On successful response, OpenTDB response_code 0 results are parsed into Question objects. Other API codes are handled as errors, including insufficient inventory, invalid parameters, session token errors and rate limiting. SocketException and TimeoutException are also surfaced as readable errors.

Question and scoring flow

Question.fromJson decodes question and answer strings through HtmlDecoder, keeps the correct and incorrect answers, combines them and shuffles the displayed choices. QuizProvider starts a 30-second Timer.periodic per question. Submission stops the timer, records the selected answer, updates correct count and streak, and appends a QuizAnswerRecord. When time expires, the provider records a timeout as an incorrect answer. The score is rounded from correct answers divided by total questions times 100.

The provider also tracks total time spent, the longest consecutive correct-answer streak in the session, per-category quiz counts and scores, a best overall percentage, recently played categories, and daily challenge completion data.

Persistence

Preference key (as used in provider)	Purpose
user_name	Player display name.
best_score_percentage	Best overall percentage.
last_category_id	Most recently selected category ID.
saved_amount / saved_difficulty / saved_type	Last quiz configuration.
favorite_categories	Favorite category IDs as strings.
recently_played_categories	Recent category IDs as strings, newest first, capped at five.
category_stats_map	JSON map of category IDs to quizzes played, best score and last score.
daily_streak_count / last_streak_date	Daily play streak value and date.
last_challenge_date / daily_challenge_score	Daily challenge completion date and score.

Persistence calls in QuizProvider use SharedPreferences. The current implementation catches storage exceptions without exposing them in the UI; persistence is best-effort.

4. Architecture, assets and configuration

Architecture and dependencies

Dependency	Observed role
flutter	Core UI framework and platform integration.
http	HTTP requests to OpenTDB.
provider	ChangeNotifierProvider, Consumer, context.watch/read for state access.
shared_preferences	Small local key-value preferences and quiz summaries.
cupertino_icons	Icon package dependency.
flutter_test / flutter_lints	Test framework and lint rules for development.

No dedicated repository layer, database package, Firebase SDK, authentication package, or custom backend client is configured.

Assets and folders

Folder	Observed contents and use
assets/branding/	Quizzical logo, icon and logo mark.
assets/categories/	Category artwork, default artwork and category-specific image files; CategoryHelper maps category IDs 9-32.
assets/images/	JPG image alternatives used by illustration/logo error fallbacks, plus category image files.
assets/quiz/	Welcome hero and configuration illustration.
assets/result/	Celebration and keep-trying illustrations.

pubspec.yaml declares all five asset directories. CategoryHelper contains unique paths for the 24 known OpenTDB categories, and tests check uniqueness of those mapped paths. A fallback visual is also available for unknown category IDs.

Platform/configuration observations

- pubspec.yaml names the package quizzical, sets version 1.0.0+1 and constrains the Dart SDK to ^3.13.4.
- Android manifest declares INTERNET permission and the Flutter launcher activity. Android Gradle configuration uses the Flutter Gradle plugin and Java/Kotlin target 17.
- The repository includes Flutter platform directories for Android, iOS, web, Linux, macOS and Windows; their existence does not by itself prove every platform has been built or runtime-tested.
- No firebase configuration files or Firebase Gradle/plugin setup were found in the inspected project configuration. Firebase integration is not implemented in the current project.
- No custom fonts are declared in pubspec.yaml.

5. Tests, limitations and viva review

Tests

test/quiz_test.dart covers HTML entity decoding, Category and Question parsing, CategoryStats serialization, category artwork uniqueness, provider defaults/configuration and favorites toggling. test/widget_test.dart checks that QuizzicalApp renders the home title and GET STARTED CTA. The repository test suite was run during this task and all 10 tests passed.

Known scope and limitations

- Internet is required to obtain current categories and quiz questions; an offline question bank is not present.
- There is no Firebase, authentication, user account, custom backend, SQL/NoSQL database, cloud backup or multi-device synchronization.
- Player identity is a locally stored display name, not an authenticated account.
- Saved progress relies on SharedPreferences and is limited to this installation/device.
- The repo contains android/, ios/, web/, linux/, macos/ and windows/ folders, but this report does not claim verified release builds for every target.

Viva questions and concise answers

Question	Answer
How is state managed?	QuizProvider extends ChangeNotifier and is registered using Provider in main.dart. Screens subscribe with watch/Consumer and call actions with read.
How does the app get questions?	TriviaService sends HTTPS GET requests to OpenTDB and converts successful JSON results into Question models.
Why decode HTML entities?	Trivia text can contain encoded punctuation/entities; HtmlDecoder turns them into readable text before rendering.
How is the answer timer implemented?	QuizProvider runs a one-second periodic Timer, updates remainingSeconds, submits a timeout when it reaches zero, and cancels the timer on answer/transition/dispose.
Where are scores and favorites stored?	SharedPreferences stores app-local values; it is not a relational database or cloud service.
Does this project use Firebase?	No. Firebase integration is not implemented in the current project.
How is the final percentage calculated?	Rounded correct-answer count divided by question count multiplied by 100.

Quick demonstration order

1. Set a player name on the welcome screen. 2. Open categories and try search/favorite filters. 3. Choose a category and configure a short quiz. 4. Demonstrate answer feedback and timer. 5. Complete the quiz and explain the result. 6. Point out locally saved stats and explain that question content comes from OpenTDB, not Firebase.

Source files reviewed

The report is grounded in pubspec.yaml, Android manifest/Gradle configuration, the Dart files listed in Section 2, assets under assets/, and tests under test/. It describes implementation rather than assuming undocumented services or features.